

SERVICIO NACIONAL DE APRENDIZAJE — SENA

REGIONAL ANTIOQUIA

CENTRO DE TECNOLOGÍA Y MANUFACTURA AVANZADA (CTMA)

TALLER PRÁCTICO: ORQUESTACIÓN CON KUBERNETES Y MINIKUBE

Informe de Evidencias Prácticas y Documentación de Infraestructura

Información del Aprendiz:

- **Nombre del Aprendiz:** Davier Andres Quinto Bejarano
- **Programa de Formación:** Analisis y Desarrollo de Software (ADSO)
 - **Número de Ficha:** 3229209
 - **Trimestre:** V

Información de la Entrega:

- **Instructor:** Richard Bentarcur
- **Centro de Formación:** Centro de Tecnología y Manufactura Avanzada (CTMA)
 - **Fecha de Entrega:** 27 de Agosto de 2026
 - **Medellín, Antioquia**

Documentación Técnica del Manifiesto YAML (**app-nginx.yaml**)

El archivo de infraestructura declarativa define dos recursos esenciales dentro del modelo de objetos de Kubernetes: un **Deployment** (encargado de gestionar los Pods y mantener la disponibilidad) y un **Service** (encargado del enrutamiento de red y balanceo de carga interno).

```
# =====
# RECURSO 1: DEPLOYMENT (Gestión de Aplicación)
# =====
apiVersion: apps/v1      # Especifica la versión de la API de Kubernetes para Deployments
kind: Deployment         # Define que el objeto creará y actualizará Pods automáticamente
metadata:
  name: nginx-deployment # Nombre del Deployment dentro del clúster
  labels:
    app: nginx           # Etiqueta organizativa principal[cite: 1]
spec:
  replicas: 3             # Mantiene constantemente 3 réplicas idénticas (Pods) en ejecución[cite: 1]
  selector:
    matchLabels:
      app: nginx          # Asocia este Deployment con los Pods etiquetados como 'app: nginx'[cite: 1]
  template:               # Plantilla base para instanciar cada uno de los Pods[cite: 1]
    metadata:
      labels:
        app: nginx        # Etiqueta asignada a cada nuevo Pod creado[cite: 1]
    spec:
      containers:
        - name: nginx      # Nombre del contenedor dentro del Pod[cite: 1]
          image: nginx:1.25 # Imagen de contenedor oficial traída desde Docker Hub[cite: 1]
          ports:
            - containerPort: 80 # Puerto interno donde la aplicación dentro del contenedor escucha
                                # peticiones[cite: 1]
---
# =====
# RECURSO 2: SERVICE (Exposición y Red)
# =====
apiVersion: v1           # Versión de la API para servicios fundamentales de red[cite: 1]
kind: Service            # Define un punto de acceso de red persistente hacia los Pods[cite: 1]
metadata:
  name: mi-servicio-nginx # Nombre del recurso Service en el clúster[cite: 1]
spec:
  type: NodePort          # Expone el servicio fuera del clúster usando un puerto del Nodo[cite: 1]
  selector:
    app: nginx            # Redirige el tráfico recibido solo a Pods con la etiqueta 'app: nginx'[cite: 1]
  ports:
    - port: 80            # Puerto de entrada interno del servicio dentro del clúster[cite: 1]
      targetPort: 80      # Puerto destino en el contenedor Nginx al que reenvía el tráfico[cite: 1]
      nodePort: 30080     # Puerto expuesto en el nodo local (rango 30000-32767)[cite: 1]
```

Resumen Explicativo de Componentes Clave

Para complementar la documentación en Word, puedes incluir esta tabla que desglosa el funcionamiento de cada propiedad:

Componente	Línea / Elemento	Función en la Arquitectura
Alta Disponibilidad	replicas: 3	Garantiza tolerancia a fallos. Si un contenedor o Pod se detiene, el <i>Control Plane</i> de Kubernetes levanta automáticamente un reemplazo para mantener las 3 réplicas[cite: 1].
Vinculación Lógica	selector: matchLabels	Asocia automáticamente el Servicio con los Pods que contienen la misma etiqueta (app: nginx), permitiendo descubrir nuevos Pods dinámicamente[cite: 1].
Mapeo de Puertos	nodePort: 30080	Mapea las peticiones desde el puerto externo del Nodo (30080) → pasa al puerto del Servicio (80) → y entrega la petición al puerto del contenedor Nginx (80)[cite: 1].

Captura 1: Inicialización del Clúster de Kubernetes

```
PS C:\Users\Usuario> minikube start
* minikube v1.38.1 en Microsoft Windows 11 Pro 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Restarting existing docker container for "minikube" ...
* Preparando Kubernetes v1.35.1 en Docker 29.2.1...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Complementos habilitados: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\Usuario> kubectl get pods
```

:

Captura 2: Estado del Despliegue y el Servicio (Terminal)

```
PS C:\Users\Usuario> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-569f95f5cb-psg2c   1/1     Running   1 (78s ago) 57m
nginx-deployment-569f95f5cb-tdlmh   1/1     Running   1 (78s ago) 57m
nginx-deployment-569f95f5cb-wfw9r   1/1     Running   1 (78s ago) 57m
```

Captura 3: Respuesta del Servidor Web (Navegador)

```
PS C:\Users\Usuario> minikube service mi-servicio-nginx --url
http://127.0.0.1:59792
! Porque estás usando controlador Docker en windows, la terminal debe abrirse para ejecutarlo
```

Captura 4:
el contenedor en Docker

☐		Name	Container ID	Image	Port(s)	Actions
☐		minikube	78aac036d659	k8s-miniku	52454:22 ↗ Show all ports (5)	   

Captura 5: Pagina con el mensaje “ Bienvenido a Nginx”



